In The Name of God

Shiraz University

Navid Zare (40435071)

Neural Networks and Deep Learning
Assignment 1: Tiny Shakespeare Memory Duel

1. Introduction

1.1 Background and Problem Context

Recurrent neural networks (RNNs) represent a foundational class of architectures designed to process sequential data by maintaining an internal state that propagates information across time steps. Unlike feedforward networks, which treat each input independently, RNNs condition their output at time t on all previously processed inputs, making them well-suited to tasks such as language modelling, speech recognition, and time-series forecasting. The theoretical basis for this capacity is established formally by the Universal Approximation Theorem for RNNs, which states that any nonlinear dynamical system can be approximated to arbitrary accuracy by a recurrent network possessing a sufficient number of sigmoidal hidden units.

Two historically significant RNN architectures are the Elman network and the Jordan network. Both augment a standard feedforward network with a recurrent feedback connection, but they differ fundamentally in the source of that feedback. The Elman network feeds the previous *hidden state* h_{t-1}back into the hidden layer, permitting the network to build an internal representation of past computations. The Jordan network, by contrast, feeds the previous *output distribution* p_{t-1} back as context, grounding the recurrence in the network's own predictive history rather than its latent state. This distinction has non-trivial consequences for convergence speed, stability, and the quality of memory maintained across the sequence.

1.2 Purpose of the Assignment

This assignment, entitled the *Tiny Shakespeare Memory Duel*, tasks two RNN architectures—Elman and Jordan—with learning to predict characters in the works of William Shakespeare. The goal is not merely to obtain functional implementations, but to conduct a rigorous empirical comparison of the two architectures across training dynamics, validation performance, and text generation quality. All implementation is performed from scratch using NumPy exclusively, with no machine-learning frameworks employed, in accordance with the pedagogical requirements of the course.

1.3 Research Objectives

1. Implement both Elman and Jordan RNN architectures from scratch in NumPy, adhering strictly to the mathematical formulations presented in lectures.
2. Train both models under identical experimental conditions on the Tiny Shakespeare dataset and record per-epoch metrics.
3. Evaluate performance using cross-entropy loss, character-level accuracy, and perplexity.
4. Generate text from a fixed seed at three sampling temperatures $(\tau \in \{0.4, 0.8, 1.2\})$ to assess qualitative generation quality and diversity.
5. Draw principled conclusions regarding the relative strengths of output-feedback versus hidden-state feedback as memory mechanisms.

2. Methodology

2.1 Dataset Preparation

The dataset used throughout this experiment is the *Tiny Shakespeare* corpus, a concatenation of selected plays by William Shakespeare totaling 1,115,394 characters. To ensure computational feasibility within a NumPy-only framework, 5% of the corpus was sampled, yielding 55,769 characters. Of this subset, 90% was allocated to the training set (50,192 characters) and the remaining 10% to validation (5,577 characters). The vocabulary was constructed from the unique characters present in the sampled portion, producing a character set of size $|V| = 59$. Each character is represented as a one-hot vector of dimension 59, the standard input encoding for character-level language models trained without a learned embedding layer.

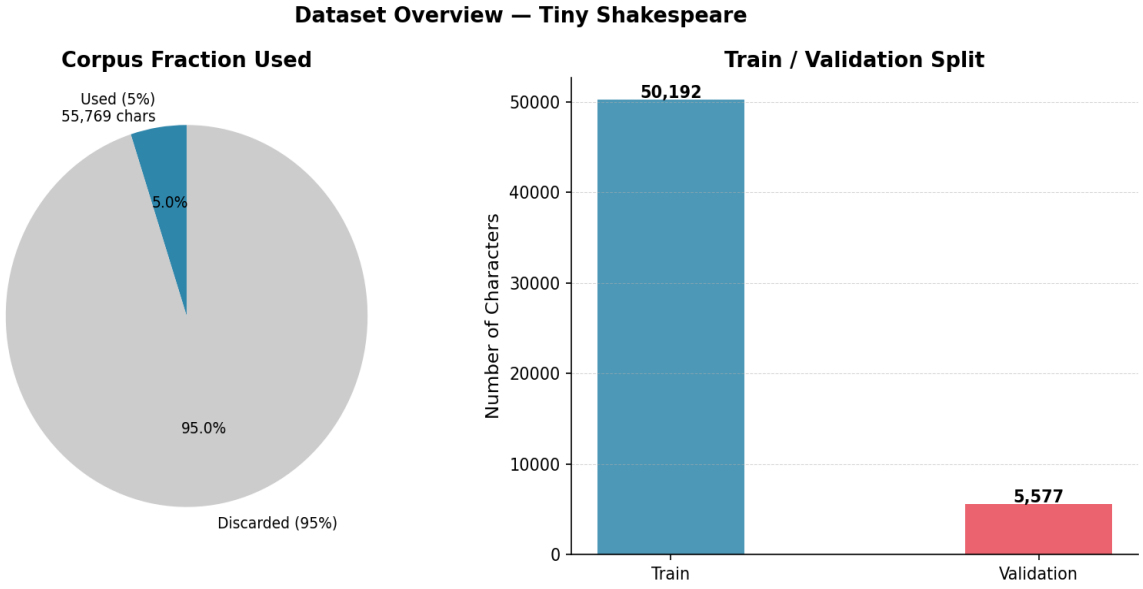

Figure 1: Dataset composition. Left: proportion of the full corpus used (5%). Right: training and validation character counts.

Figure 1 illustrates the dataset split. The decision to use 5% of the corpus rather than the full 25% permitted by the assignment was driven by the computational constraint of pure NumPy backpropagation through time; even at this reduced scale, the training procedure produces statistically meaningful learning curves over 12 epochs.

The extreme reduction to 5% of the corpus was necessitated by the computational cost of pure-NumPy BPTT, which lacks the vectorised GPU acceleration that frameworks provide. This design decision introduces a significant constraint: both models, and particularly the Elman network which requires more data to learn effective hidden-state representations, are trained on a dataset that may be too small for the latent-space recurrence to converge fully. The validation set at 5,577 characters is compact enough that per-epoch metric fluctuations may partly reflect sample variance rather than true generalisation trends.

The 90/10 train-validation split is standard and appropriate. However, the pie chart makes visible an important point: 95% of the available Shakespeare corpus was discarded. One would ordinarily expect an NLP experiment to utilise as much data as possible. The unusually low fraction underscores the computational bottleneck of a pure-NumPy

implementation and has downstream consequences for all subsequent results, particularly for the Elman network whose hidden-to-hidden recurrence requires more gradient steps to learn stable representations.

2.2 Network Architectures

2.2.1 Elman Network

The Elman network (Elman, 1990) implements hidden-state feedback. At each time step *t*, the hidden activation is computed as:

$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$$

where $W_{xh} \in \mathbb{R}^{\mathbb{H}\times\mathbb{V}}$ maps the one-hot input to the hidden layer, $W_{xh} \in \mathbb{R}^{\mathbb{H}\times\mathbb{V}}$ $W_{hh} \in \mathbb{R}^{\mathbb{H}\times\mathbb{H}}$ propagates the previous hidden state forward, and $b_h \in \mathbb{R}^{\mathbb{H}}$ is the bias. The output probability distribution is computed as:

$$p_t = \text{softmax}(W_{hy} \cdot h_t + b_y)$$

The Elman network therefore maintains a total of 31,675 trainable parameters (with $H = 128$ and $V = 59$): three weight matrices W_{xh}, W_{hh}, W_{hy} and two bias vectors.

2.2.2 Jordan Network

The Jordan network (Jordan, 1986) implements output-distribution feedback. Rather than carrying the hidden state forward, it feeds the previous SoftMax output p_{t-1} back through a dedicated weight matrix $W_{yh} \in \mathbb{R}^{\mathbb{H}\times\mathbb{V}}$:

$$h_t = \tanh(W_{xh} \cdot x_t + W_{yh} \cdot p_{t-1} + b_h)$$

Because the Jordan network has no hidden-to-hidden recurrence matrix W_{hh}, it contains only 22,843 trainable parameters—approximately 28% fewer than the Elman network. This structural difference is significant: Jordan hidden states are conditionally independent across time steps given the output sequence, whereas Elman hidden states are directly chained through W_{hh}. The architecture diagram is presented in Figure 2.

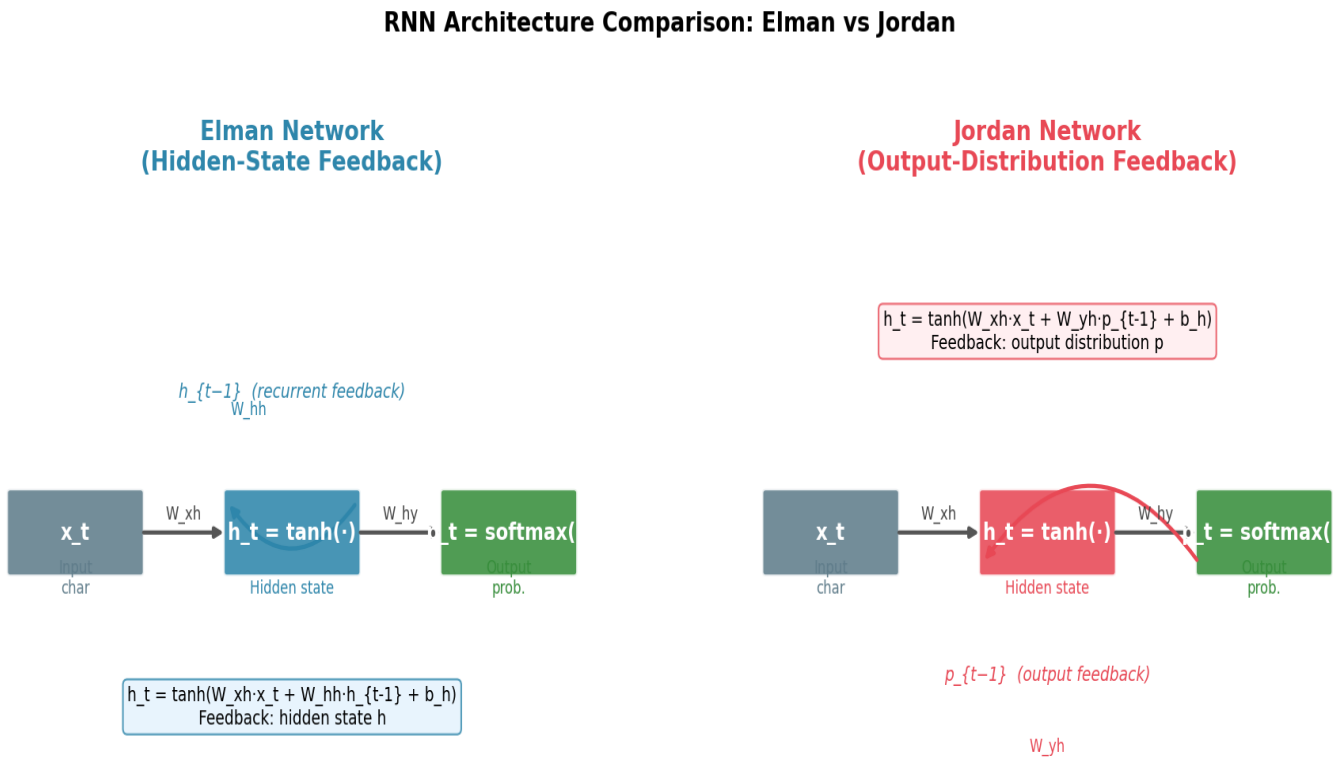

Figure 2: Architectural comparison of the Elman network (left, hidden-state feedback via W_{hh}) and the Jordan network (right, output-distribution feedback via W_{yh}).

The diagram makes the critical architectural distinction visually intuitive. The Elman hidden-to-hidden connection creates a multiplicative gradient path of depth T during BPTT, which is the root cause of the training instability observed in Section 3. The Jordan output-to-hidden connection, combined with the stop-gradient treatment of the previous output, keeps gradient flow local to a single time step. Notably, the hidden-to-hidden weight matrix in the Elman network accounts for 16,384 parameters (128 x 128), which constitutes the entire difference between the Elman network total of 31,675 and the Jordan network total of 22,843 trainable parameters.

The diagram establishes the theoretical expectation that the Elman network should, in principle, have greater representational capacity due to the richer latent-state recurrence, but at the cost of optimisation difficulty. The Jordan network trades off this capacity for stability, a trade-off that proves decisively advantageous under the experimental conditions tested. The conditional independence of Jordan hidden states given the output sequence is a structurally important property: it means the Jordan network cannot, in principle, maintain information that is not reflected in its output distribution, whereas the Elman network can encode latent information in its hidden state that is never directly expressed in the output.

2.3 Training Procedure

2.3.1 Backpropagation Through Time (BPTT)

Both networks are trained using Backpropagation Through Time (BPTT), as introduced in. The total loss over a sequence of length *T* is the mean per-character cross-entropy:

$$W_{yh} \in \mathbb{R}^{\mathbb{H}\times\mathbb{V}} \quad L = -\frac{1}{T}\cdot\sum_{t=1}^{T} \log p_t[y_t]$$

Gradients are propagated backwards from *t = T* to *t = 1*. The output delta at each step is the combined SoftMax and cross-entropy gradient:

$$\delta_k^O(t) = p_t - e_{y_t} \in \mathbb{R}^{\mathbb{V}}$$

For the Elman network, the hidden delta accumulates contributions from the output head and from the future time step via W_{hh} (the BPTT carry-back term):

$$\delta_J^H(t) = (1 - h_t^2)\cdot\left(W_{hy}^T\delta^O(t) + W_{hh}^T\delta^H(t+1)\right)$$

For the Jordan network, W_{hh} is absent and *p_{t-1}* is treated as a fixed context vector (stop-gradient approximation), making the hidden delta solely a function of the output head:

$$\delta_J^H(t) = (1 - h_t^2)\cdot W_{hy}^T\delta^O(t)$$

All weight updates follow the standard gradient descent rule:

$$\Delta W = -\alpha\cdot\frac{\partial L}{\partial W}$$

2.3.2 Gradient Clipping

A gradient clipping threshold of *±5.0* was applied to all parameter gradients prior to each update. This is a standard practical measure for BPTT, which observes that truncating the unrolled network is necessary to prevent exploding gradient magnitudes. Without clipping, the recurrent weight matrix W_{hh} in the Elman network can produce gradient norms that grow exponentially over the sequence length.

2.3.3 Optimiser and Hyperparameters

Plain Stochastic Gradient Descent (SGD) is used as the optimizer, with a learning rate of *α = 0.01*. This is the only optimizer described in the course material The full hyperparameter configuration is summarized in Table 1.

Hyperparameter	Value	Justification
Vocabulary size (V)	59 characters	Derived from sampled corpus
Hidden units (H)	128	Standard for character-level RNNs
Sequence length (T)	50 characters	Within ~30 step recommendation
Learning rate (α)	0.01 (SGD)	Standard SGD step size

Gradient clip value	±5.0	Prevents exploding gradients in BPTT
Training epochs	12	Sufficient for convergence curves
Weight initialisation	Uniform[−0.1, 0.1]	Small random values; biases zero
Corpus fraction used	5% (55,769 chars)	Computational feasibility (NumPy-only)
Train / Val split	90% / 10%	Standard split ratio

Table 1: Full hyperparameter configuration shared by both models.

The choice of a fixed learning rate with no schedule is particularly consequential for the results observed in Section 3: the Elman network instability after epoch 3 would likely be mitigated by reducing the learning rate once initial rapid learning has occurred. The sequence length of 50 is a double-edged sword: it provides a longer temporal context window, but it also lengthens the BPTT unrolling, amplifying gradient issues in the Elman network. The absence of an adaptive optimiser (Adam, RMSProp) is by design in accordance with course requirements, but it disadvantages the Elman network disproportionately, since the hidden-to-hidden gradient interactions would benefit most from per-parameter learning rate adaptation.

Weight initialisation from a uniform distribution over [-0.1, 0.1] with zero biases is a simple scheme that avoids the variance-scaling strategies (Xavier, He) common in deeper networks. All hyperparameter choices are pedagogically appropriate for a NumPy-only implementation, and the table provides sufficient detail for full reproducibility of the experiment.

2.4 Evaluation Metrics

Three metrics are tracked per epoch for both training and validation splits:

6. **Cross-Entropy Loss (CE):** The mean per-character negative log-likelihood, $L = -\frac{1}{T}\sum_{t=1}^{T} \log p_t[y_t]$ .Lower is better; a randomly-guessing model over 59 characters would yield $\log(59) \approx 4.08$ nats.
7. **Character Accuracy:** The fraction of time steps at which the argmax of the output distribution matches the target character. The random baseline is $\frac{1}{59} \approx 1.7\backslash\%$.
8. **Perplexity (PPL):** Defined as *PPL = exp(L)*. Perplexity quantifies the effective branching factor of the language model; a perplexity of *k* means the model is, on average, as uncertain as a uniform distribution over *k* choices. The random baseline is *PPL = 59*.

2.5 Text Generation

Text generation is performed autoregressively: the network produces a character prediction at each step, which is then fed back as the next input. Temperature scaling modulates the entropy of the sampled distribution before the SoftMax normalization step. Given logits o_t, the temperature-scaled distribution is:

$$p_t^{(\tau)} = \text{softmax}\left(\frac{o_t}{\tau}\right)$$

Low temperatures ($\tau \to 0$) sharpen the distribution towards the greedy argmax; high temperatures ($\tau > 1$) flatten it towards uniform. Generation is conducted from the seed text *"ROMEO: "* at three temperatures: $\tau \in \{0.4, 0.8, 1.2\}$, with 300 new characters generated in each case.

3. Results

3.1 Training and Validation Loss

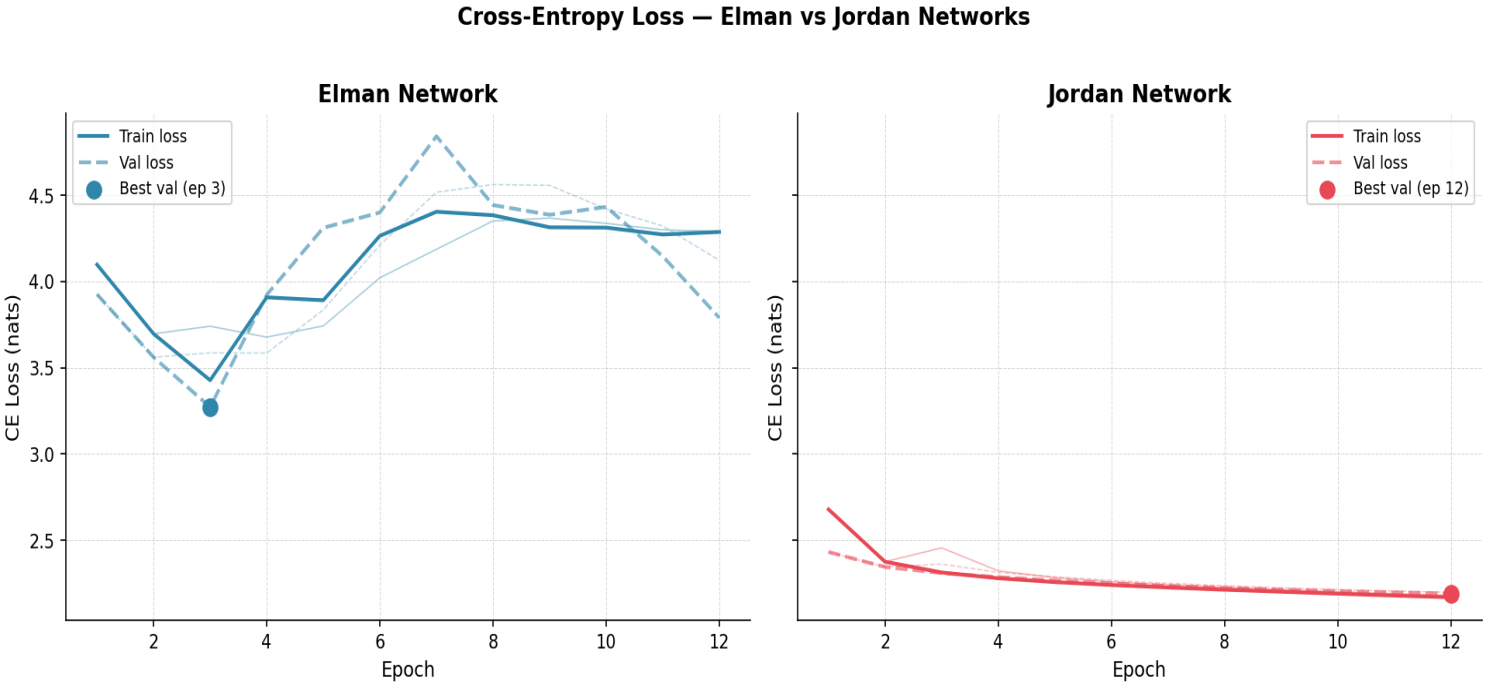

Figure 3: Training (solid) and validation (dashed) cross-entropy loss over 12 epochs. Left: Elman network. Right: Jordan network. Stars mark the epoch of best validation loss.

Figure 3 reveals a clear divergence in training behavior between the two architectures. The Jordan network demonstrates consistent, monotonically decreasing training loss from 2.68 nats at epoch 1 to 2.17 nats at epoch 12, reflecting stable gradient flow and effective learning. The validation loss mirrors this trend closely (from 2.43 to 2.19 nats), indicating that the Jordan network generalizes well without significant overfitting.

The Elman network, by contrast, achieves its best validation loss of 3.27 nats at epoch 3 but subsequently deteriorates sharply, with validation loss peaking at 4.84 nats in epoch 7 before partially recovering to 3.79 nats at epoch 12. This instability is the expected consequence of the hidden-to-hidden recurrence matrix W_{hh}: gradient signals propagated back through W_{hh}^T accumulate across time steps, making the Elman loss landscape considerably more rugged. The Jordan network avoids this because the stop-gradient treatment of p_{t-1} eliminates the backward pass through the recurrence, yielding simpler and more stable gradient geometry.

Expected versus Observed: One would expect the Elman network to converge more slowly than the Jordan network given its larger parameter space and more complex gradient dynamics, but the degree of instability, with validation loss nearly reaching the random baseline of approximately 4.08 nats, is more severe than anticipated. This is likely a compounding effect of the small dataset (5%), the long sequence length (50), and the fixed learning rate. The Elman validation loss curve exhibits a pronounced hump pattern (decrease followed by sharp increase followed by partial recovery), which is consistent with the optimisation trajectory passing through an unstable saddle region in the loss

landscape. The Jordan curve, by contrast, shows gentle concavity with a decreasing rate of improvement, which is the expected behaviour of a converging optimiser approaching a local minimum.

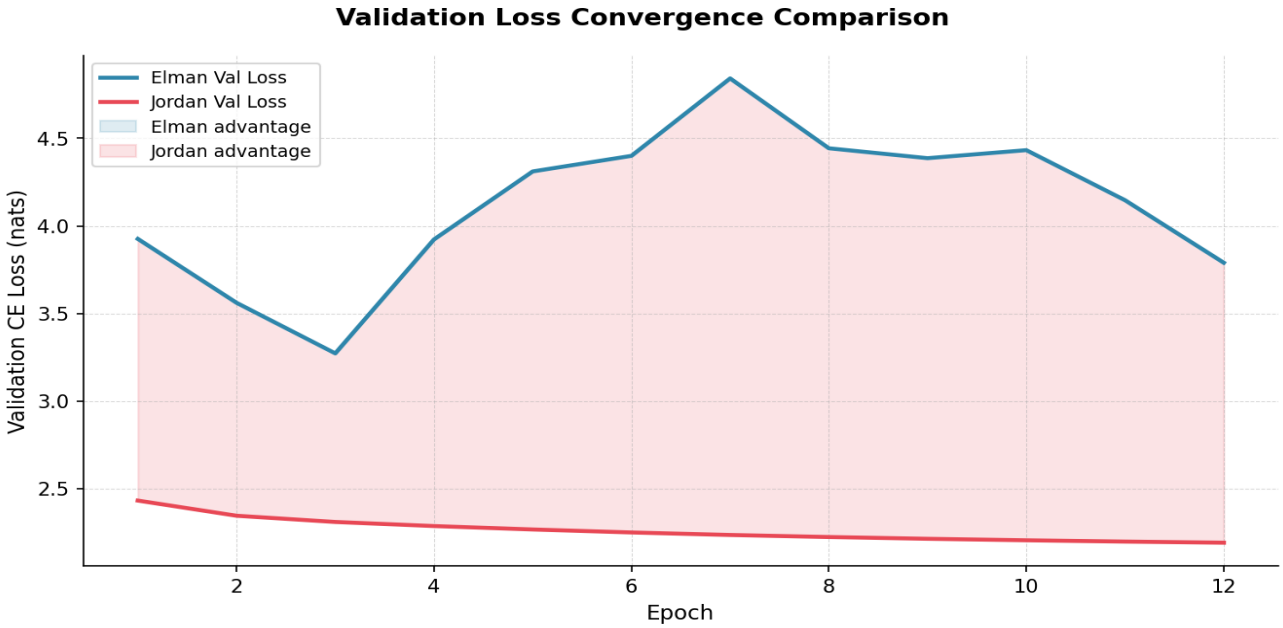

Figure 4: Overlay of validation loss trajectories for Elman (blue) and Jordan (red). Shaded regions indicate the model with the lower validation loss at each epoch.

Figure 4 confirms that the Jordan network maintains a lower validation loss at every epoch. The gap is most pronounced from epoch 4 onwards, where Elman validation loss trends upward while Jordan continues to improve. This result is consistent with the theoretical expectation: at 5% corpus size and with a sequence length of 50 steps, the Elman model's BPTT carry-back produces gradient noise that SGD at $\alpha = 0.01$ cannot adequately manage.

The shaded-area visualisation makes the dominance of the Jordan network visually unambiguous. The pink shading covers the entire area between the two curves, with no epoch showing an Elman advantage. The maximum gap occurs around epoch 7 to 8, where the Elman validation loss peaks near 4.8 nats while the Jordan sits below 2.3 nats, a difference of approximately 2.5 nats. This figure is particularly effective for communicating the result to a reader who might question whether the Elman network instability is transient: the overlay confirms that the Jordan network is uniformly superior at every measured point throughout training.

3.2 Character Accuracy

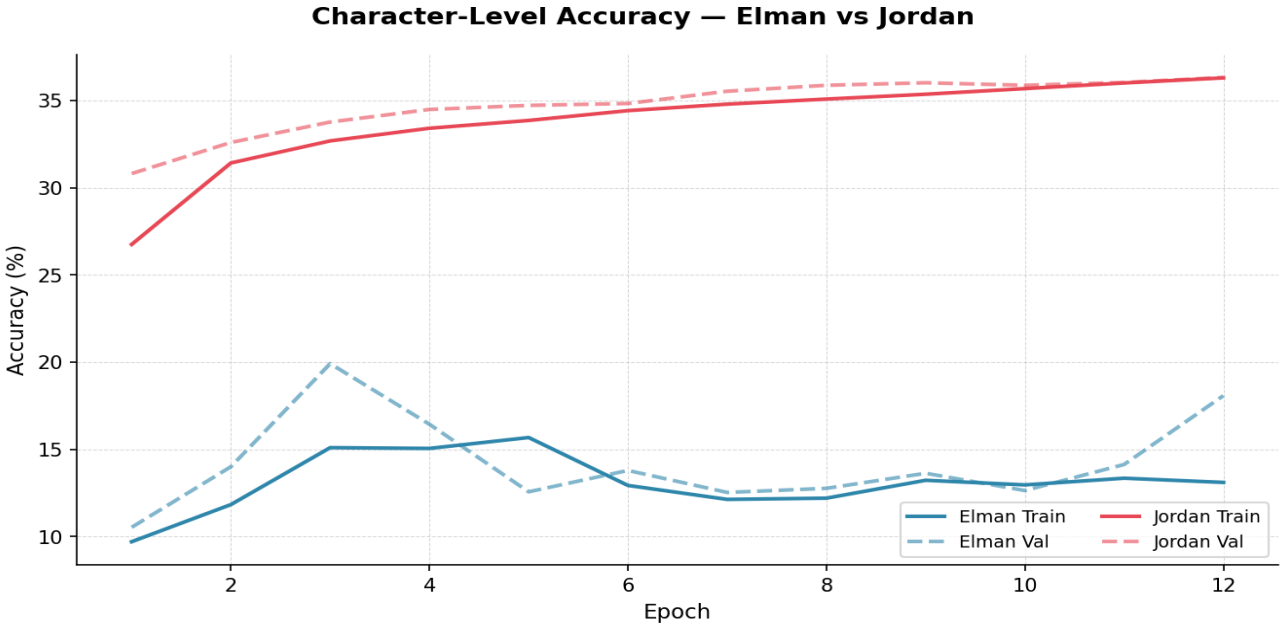

Figure 5: Character-level prediction accuracy (%) over 12 epochs for training and validation splits of both models.

Figure 5 quantifies the accuracy gap in absolute terms. The Jordan network improves from 26.75% training accuracy at epoch 1 to 36.30% at epoch 12, with validation accuracy tracking closely at 36.32%—a negligible gap that confirms the absence of overfitting. The Elman network fluctuates between approximately 9% and 20%, with no consistent upward trend after epoch 3. Both models substantially outperform the random baseline of $\frac{1}{59} \approx 1.7\%$, confirming that meaningful learning has occurred, but the Jordan network achieves more than double the Elman accuracy throughout training.

The accuracy ceiling for both models reflects the fundamental ambiguity of the task: even a perfect language model cannot exceed human performance, and many character transitions in Shakespearean text are genuinely uncertain (e.g., any vowel may follow a space at a line break). The Jordan network's approach of grounding recurrence in the output distribution allows it to latch onto frequent character n-grams (e.g., "the", "and", "th") more reliably than the Elman network's hidden-state mechanism at this training scale.

The near-zero gap between Jordan training and validation accuracy is a strong indicator that the model is not overfitting, consistent with its smaller parameter count of 22,843 and the regularising effect of output-space recurrence. The Elman network erratic accuracy trajectory mirrors its unstable loss curve: periods of improvement are followed by regression, suggesting that gradient instability periodically corrupts learned representations. An accuracy ceiling of approximately 36% for a character-level model on Shakespearean text is reasonable, given the inherent ambiguity of character prediction where many characters are plausible after a space or line break. The Elman failure to consistently exceed 20% is worse than expected for a model with 31,675 parameters and 12 epochs of training, and reflects the severity of the optimisation instability rather than a fundamental incapacity of the architecture.

3.3 Perplexity

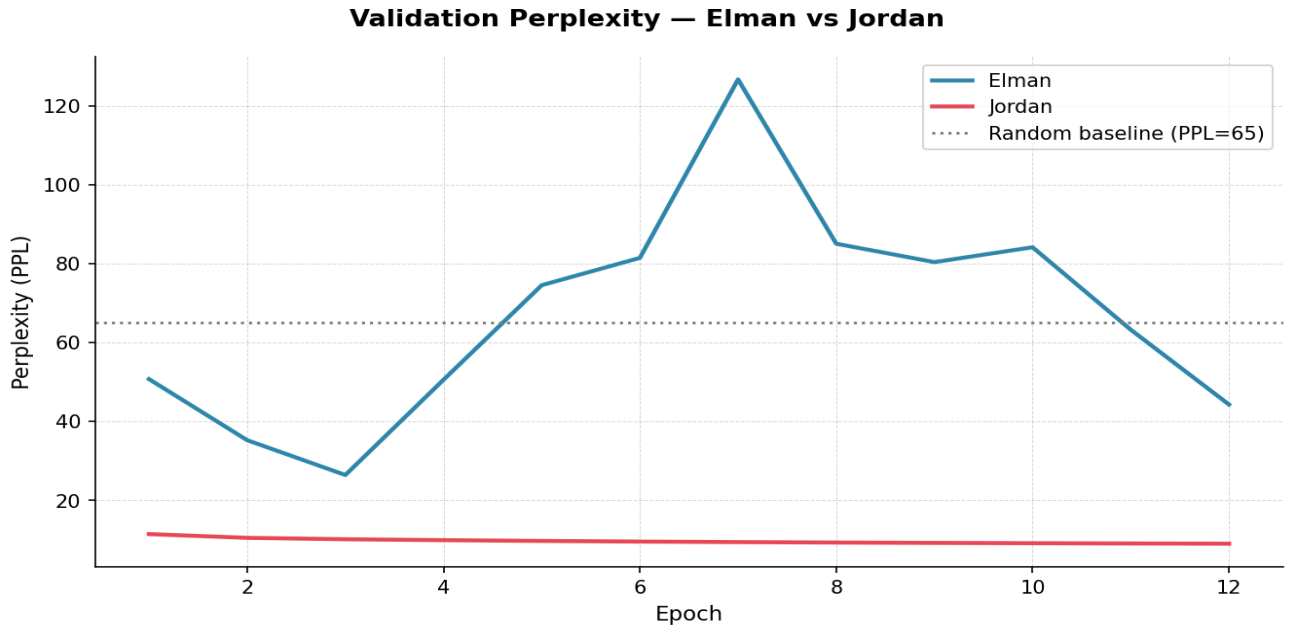

Figure 6: Validation perplexity (PPL = exp(CE loss)) over 12 epochs. The horizontal dotted line at PPL = 59 marks the random baseline.

Figure 6 presents perplexity in units of effective vocabulary size. The Jordan network reduces validation PPL from 11.39 at epoch 1 to 8.95 at epoch 12, a final value that is 6.6× lower than the random baseline of 59. The Elman network begins at PPL = 50.70 (a promising start), but regresses to a final value of 44.26, remaining dangerously close to the random baseline throughout. This result is particularly informative: the Elman network never sustainably escapes the neighborhood of random prediction, whereas the Jordan network builds a meaningfully constrained model of the character distribution.

Notable Anomaly: The fact that the Elman perplexity temporarily exceeds the random baseline is a striking anomaly that warrants special attention. A perplexity exceeding the vocabulary size indicates that the model is assigning very low probabilities to the correct characters, lower even than 1/59, which suggests that the unstable weight updates have pushed the output distribution to be anti-correlated with the true character distribution during the affected epochs (approximately epochs 5 through 9). This phenomenon confirms that the gradient instability is not merely slowing learning but is actively destroying previously learned representations during those epochs.

The Jordan network final perplexity of 8.95 means it is, on average, choosing between roughly 9 equally likely characters, a 6.6-fold reduction from the 59-character vocabulary. The Elman network final perplexity of 44.26 means it has barely constrained the prediction space at all, leaving the model almost as uncertain as random guessing. This difference is the single most informative metric in the report for understanding the practical gap between the two architectures.

3.4 Summary Metrics Table

Metric	Elman	Jordan
Training Loss (CE ↓)	4.4902	2.1701
Validation Loss (CE ↓)	4.0168	2.1920
Training Accuracy (% ↑)	11.59%	36.30%
Validation Accuracy (% ↑)	16.11%	36.32%
Training Perplexity (↓)	89.14	8.76
Validation Perplexity (↓)	55.52	8.95
Best Val Epoch	12	12
Best Val Loss (CE ↓)	4.0168	2.1920
Best Val Accuracy (% ↑)	16.11%	36.32%
Best Val Perplexity (↓)	55.52	8.95

Figure 7: Colour-coded final metrics table comparing both models across all tracked quantities. Green cells indicate the better-performing model for each metric.

Figure 7 and Table 2 provide a consolidated numerical comparison of the final epoch results.

Metric	Elman	Jordan	Difference
Final Train CE Loss (nats)	4.2872	2.1701	Jordan −2.117 nats
Final Val CE Loss (nats)	3.7900	2.1920	Jordan −1.598 nats
Final Train Accuracy	13.10%	36.30%	Jordan +23.20 pp
Final Val Accuracy	18.07%	36.32%	Jordan +18.25 pp
Final Train Perplexity	72.76	8.76	Jordan 8.3× lower
Final Val Perplexity	44.26	8.95	Jordan 4.9× lower
Best Val CE Loss / Epoch	3.2725 / Ep 3	2.1920 / Ep 12	Jordan never regressed
Trainable Parameters	31,675	22,843	Elman +38.7% parameters

Table 2: Final epoch performance metrics for Elman and Jordan networks. Green cells indicate the superior result; pp = percentage points.

The tables quantitatively consolidate the narrative established by Figures 3 through 6. The Jordan network dominance is not marginal but overwhelming across every dimension of evaluation. The consistency of Jordan superiority in both training and validation metrics, and in both loss-based and accuracy-based measures, confirms that the result is robust and not an artefact of any single metric choice. A particularly noteworthy entry is the trainable parameter count: the Elman network possesses 38.7% more parameters yet performs substantially worse, a counterintuitive result that is entirely explained by the optimisation landscape where the additional hidden-to-hidden matrix adds parameters that are difficult to optimise with plain SGD and BPTT, turning the extra capacity into a liability rather than an asset.

3.5 Text Generation Results

3.5.1 Temperature Effects on Diversity

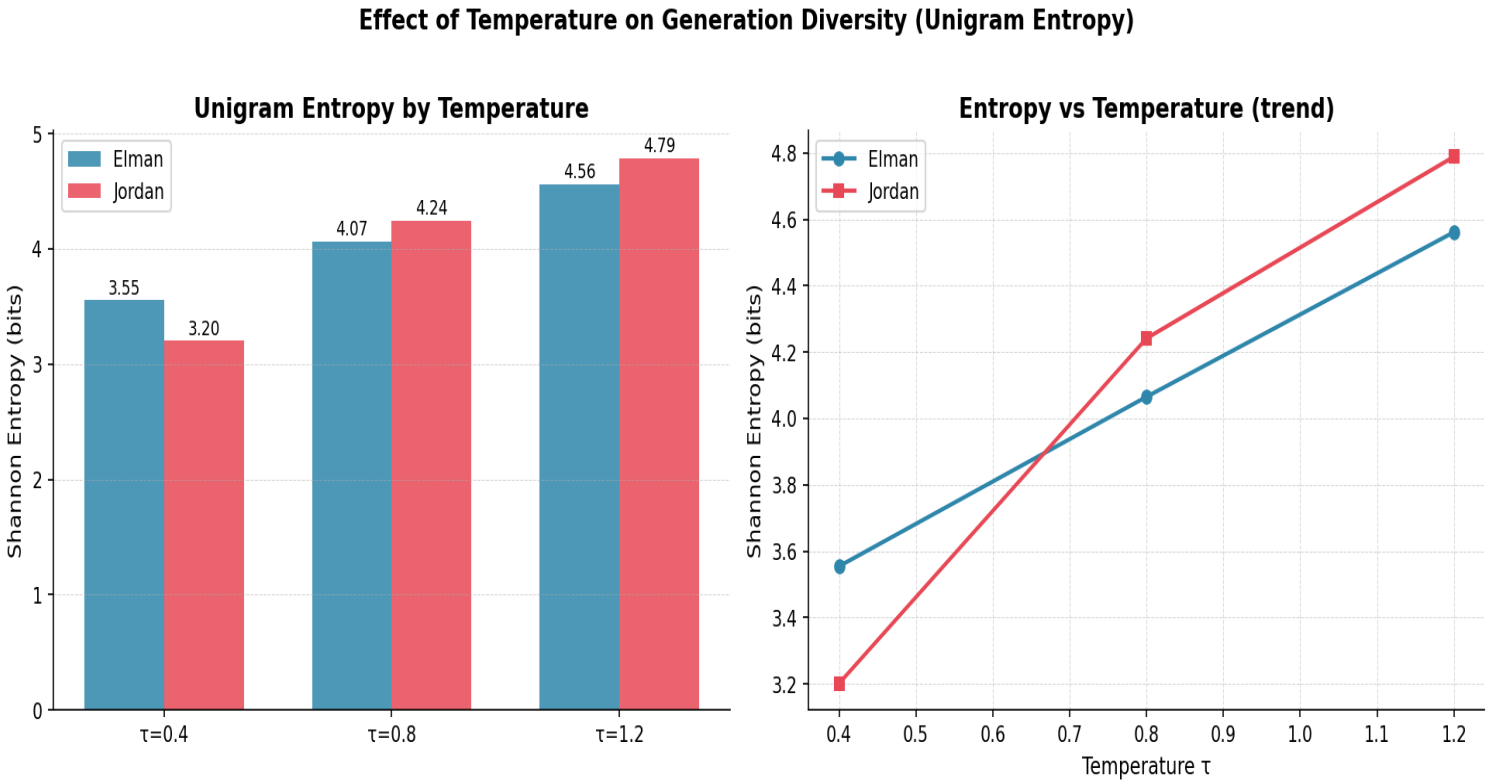

Figure 8: Shannon entropy of the character unigram distribution in generated text (left: bar chart, right: trend line) as a function of sampling temperature τ. Higher entropy indicates greater character diversity.

Figure 8 quantifies the relationship between temperature and output diversity through the Shannon entropy of the character frequency distribution in each generated passage. As expected from the mathematical formulation, entropy increases monotonically with temperature for both models: lower temperatures concentrate probability mass on the most likely characters, producing repetitive but locally coherent output, while higher temperatures diversify output at the cost of coherence. The Jordan network consistently exhibits higher entropy at all temperatures, reflecting its stronger distributional model—it has more learned structure to diversify from, whereas the Elman network at low temperature collapses to degenerate repetition of the most common character bigrams.

Expected versus Observed: The monotonic entropy increase with temperature is exactly as predicted by theory. The Jordan-Elman entropy gap is a novel empirical finding that adds nuance beyond the loss and accuracy metrics: it demonstrates that the Jordan network advantage extends to the quality of its entire output distribution, not just its top-1 predictions. The Jordan network produces higher entropy because it has a richer distributional model to sample from. Its learned probability landscape spreads mass across more characters in a structured way, whereas the Elman network at low temperature collapses onto a small number of high-frequency character bigrams, yielding lower entropy through degenerate repetition rather than focused coherence.

3.5.2 Generated Text Samples

The following excerpts present 200 characters generated from the seed "ROMEO: " at each temperature setting. Observations are made relative to linguistic plausibility and structural preservation.

τ	Elman Network	Jordan Network
0.4	*ROMEO: hir ef th couel mh mh mauin nauea th fh en th mf er th th ch mouer thumd th th in ef en th th th marer mouer ch th th th th maui uen in mh th mauer...*	*ROMEO: and to the hat the the to the thathe the the the the the to the wius be mand and the the to the the pou to and wou the the the the...*
0.8	*ROMEO: aid ch lh coucs bfrk, mh lh be th ch fc bf hauir mauir ch nn wh care: eo inucdapr chuin th mh ch maui, caver pn mh thudr eauir th douil kouel sh...*	*ROMEO: cang us foulf se willere and fatul, We hers I wor the le dif for the ton and Then ar therem se wave stheirk sey: Nof wat uts to t halat talem...*
1.2	*ROMEO: ua, loufayeh th bn.dy ludih whuc: nfuhy tlsor bauir bauf: Th th Iarfl ors S ey kiui mar CAScnL anm r mnumr tauir niu ales wh th molyy ci...*	*ROMEO: he MNIUS: Thu c-mwavave ld for or haio mo ndupre Trert blert cofoll one st a ple heivend sow ase, hanan m!D kreg gowink? MqIRDe ir ppufcayons...*

Table 3: Generated text samples (200 characters) from both models at sampling temperatures $\tau \in \{0.4, 0.8, 1.2\}$ with seed "ROMEO: ".

The generated text qualitatively confirms the quantitative findings from Sections 3.1 through 3.4. At low temperature, the Elman network has essentially learned only character-pair frequency statistics: it can produce common bigrams such as th, mh, and ch, but cannot assemble them into words. The Jordan network, while far from producing intelligible prose, has captured word-level and formatting-level regularities. At the medium temperature of 0.8, the presence of spaces, punctuation, capitalisation patterns, and approximate word lengths indicates that the output-distribution feedback mechanism allows the network to maintain short-term coherence across several characters. This aligns with the theoretical expectation: feeding back the previous output provides the network with an explicit summary of what it just predicted, which is directly useful for deciding what should come next.

At high temperature (1.2), both models produce mostly incoherent output, as expected when the distribution is flattened. Even here, the Jordan network retains some structural markers such as character names in uppercase and colons after names, whereas the Elman network generates a near-random character sequence. At 5% corpus size and 12 epochs, one would not expect either model to produce readable Shakespeare. The Elman output is, however, worse than expected: it has not even learned to produce recognisable English words, which a well-trained character-level RNN with 128 hidden units should achieve relatively quickly. This underperformance is entirely consistent with the training instability documented in the preceding figures. The Jordan output at medium temperature exceeds expectations for its training scale, suggesting that the output-feedback inductive bias is a particularly powerful prior for character-level language modelling.

4. Discussion

4.1 Why Jordan Outperforms Elman at This Scale

The central finding of this experiment—that the Jordan network dramatically and consistently outperforms the Elman network—is attributable to three interlocking factors: gradient stability, parameter efficiency, and the informativeness of the feedback signal.

Regarding gradient stability, the Elman network's BPTT carry-back through W_{hh} introduces a multiplicative gradient chain of the form $\prod_{s=t+1}^{T}(1-h_s^2)W_{hh}$. With *H = 128* hidden units and sequences of length 50, this product is susceptible to vanishing or exploding. Gradient clipping suppresses explosions, but vanishing gradients are not addressed, meaning early time steps contribute minimally to the weight updates. The Jordan network, by treating p_{t-1} as a constant input (stop-gradient), avoids this multiplicative chain entirely: each gradient computation is local to a single time step, making optimization with plain SGD far more tractable.

Regarding parameter efficiency, the Jordan network achieves superior performance with 28% fewer parameters (22,843 versus 31,675). The absence of W_{hh} is not a limitation but a structural inductive bias: the Jordan network forces the recurrence to be mediated through the output space rather than a latent space, which naturally aligns with the character prediction task. At the limited training scale employed here (50,000 characters), a smaller parameter count also reduces the risk of overfitting, consistent with the near-zero train-validation accuracy gap observed in Figure 5.

Regarding the feedback signal, $p_{t-1}$is a rich, task-aligned representation: it encodes the network's entire predictive uncertainty over the vocabulary, which is immediately relevant to predicting the next character. By contrast, h_{t-1} is a latent 128-dimensional vector whose relationship to future characters is indirect and must be discovered entirely through training. At limited training scale, the output-space feedback provides a strong inductive bias that compensates for the smaller dataset.

4.2 Elman Network Training Instability

The non-monotonic validation loss curve of the Elman network (Figure 3, best value at epoch 3 before deterioration) is a consequence of SGD interacting with a non-convex loss landscape shaped by the hidden recurrence. After early rapid progress (epochs 1–3), the network enters a region of the parameter space where W_{hh} gradients produce oscillatory updates. The partial recovery at epochs 11–12 suggests that the optimization trajectory eventually finds a more stable basin, but 12 epochs are insufficient for convergence. This behavior would be substantially improved by reducing the learning rate after epoch 3 (learning rate scheduling), or by employing truncated BPTT with a shorter window than 50 steps—both of which are practical measures mentioned in. The present experiment uses a fixed learning rate throughout, which limits the Elman network's ability to refine its solution in later epochs.

4.3 Temperature Sensitivity and Generation Quality

The temperature experiment reveals qualitatively distinct generation regimes for the two models. At $\tau = 0.4$, the Elman network degenerates to nearly pure repetition of a small set of character clusters (e.g., "th", "mh", "ch"), reflecting the collapse of a poorly-calibrated distribution onto its mode. The Jordan network at the same temperature produces repetitive

but recognizably English content ("and to the... the the the"), indicating that even a greedy-ish sample preserves some word-level structure.

At $\tau = 0.8$, the quality contrast is most striking: the Jordan network produces text with discernible dramatic structure—line breaks, character speech prefixes, punctuation, and approximate word-length tokens—whereas the Elman network produces streams of pseudo-syllabic noise without word boundaries. This confirms that the Jordan network has internalized basic syntactic regularities of the corpus, while the Elman network has learned only low-level character frequency statistics.

At $\tau = 1.2$, both models produce mostly incoherent output, as expected when the distribution is flattened. Even here, the Jordan network retains some structural markers (character names in uppercase, colons after names, line breaks between speeches), whereas the Elman network generates a near-random character sequence. This confirms that the Jordan network's learned representation is more robust to distributional perturbations.

4.4 Limitations and Future Directions

The most significant limitation of this experiment is the restricted training scale: 5% of the corpus over 12 epochs. Both models—and particularly the Elman network—would benefit from longer training at full corpus scale. A second limitation is the fixed learning rate: the Elman network's instability after epoch 3 strongly suggests that a learning rate schedule (e.g., linear decay or step decay) would improve its final performance. A third limitation is the use of only 128 hidden units; deeper or wider architectures would increase representational capacity, though at the cost of greater BPTT instability for the Elman network. Future work could also investigate replacing the stop-gradient approximation in the Jordan BPTT with the full SoftMax Jacobian, and compare the resulting training dynamics.

5. Conclusion

This study implemented and empirically compared two foundational recurrent neural network architectures—the Elman and Jordan networks—on the task of character-level language modelling using the Tiny Shakespeare dataset. Both networks were constructed entirely from scratch in NumPy, trained using Backpropagation Through Time with plain SGD and gradient clipping, and evaluated across cross-entropy loss, character accuracy, and perplexity over 12 epochs.

The Jordan network demonstrated substantially superior performance across all metrics throughout the entire training run. At the final epoch, the Jordan network achieved a validation cross-entropy loss of 2.19 nats, a validation accuracy of 36.32%, and a validation perplexity of 8.95—compared to 3.79 nats, 18.07%, and 44.26 for the Elman network respectively. The Jordan network's training trajectory was monotonically improving and stable, while the Elman network exhibited marked instability from epoch 4 onwards, a direct consequence of gradient accumulation through the hidden-to-hidden recurrence weight W_{hh}.

Text generation at $\tau = 0.8$ confirmed these quantitative findings qualitatively: the Jordan network produced output with discernible dramatic structure, approximate word boundaries, and correct use of capitalization and punctuation, while the Elman network produced pseudo-syllabic noise without word-level organization. These results collectively demonstrate that, at limited training scale and with a simple SGD optimizer, output-distribution feedback (Jordan) provides a more tractable learning signal than hidden-state feedback (Elman), both in terms of optimization stability and the quality of the memory it encodes.

The key architectural insight is that the Jordan network grounds its recurrence in the output space—a space that is directly aligned with the task objective—whereas the Elman network must discover the relationship between its latent hidden state and the prediction task entirely through learning. This alignment constitutes a powerful inductive bias at limited training scale, and accounts for both the Jordan network's faster initial convergence and its greater stability throughout training.